Banker's Algorithm

Deadlock Avoidance and Detection

Course Project Report

Operating Systems | Usman Institute of Technology, Karachi

Name	Roll Number
M. Ahsan	24FA-059-CS
M. Ahmad	24FA-050-CS
Umair Jan	24FA-009-CS
Umar Sani	24FA-038-CS

Contents

1. Project Title and Objective

Project Title: Banker's Algorithm — Deadlock Avoidance and Detection

The objective of this project is to design and implement a fully interactive simulator of the Banker's Algorithm, a classical deadlock avoidance algorithm in Operating Systems. The simulator allows users to define a system with any number of processes and resource types, input Maximum Demand and Allocation matrices, submit runtime resource requests, and observe the system's safety status in real time. The project bridges theoretical OS knowledge with a practical, visual implementation built entirely in Python.

2. Problem Statement

In a multi-process operating system, multiple processes compete for a shared pool of finite resources such as memory, I/O devices, and CPU time. A critical hazard that can arise in such environments is deadlock, which is a situation where a set of processes are each waiting for resources held by another process in the set, and no process can proceed. Deadlock results in the complete halt of all affected processes, wasting system resources and requiring manual intervention to recover.

The Banker's Algorithm, proposed by Edsger Dijkstra, addresses this problem through the strategy of deadlock avoidance. Rather than allowing resource requests to be granted freely, the algorithm evaluates each request against the current system state and only grants it if the resulting state remains safe — meaning there exists at least one sequence in which all processes can complete without entering deadlock.

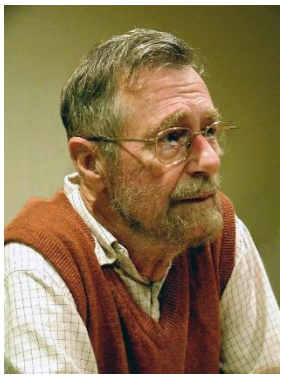


Figure 1: Edsger W. Dijkstra

This project simulates that decision-making process in an interactive graphical environment, making the algorithm visible and understandable.

3. Applied Operating System Concepts

3.1 Deadlock

Deadlock is a state in which a set of processes are permanently blocked because each is waiting for a resource held by another. Four conditions must simultaneously hold for deadlock to occur:

- Mutual Exclusion: Resources cannot be shared; only one process may use a resource at a time.
- Hold and Wait: A process holds at least one resource while waiting for additional resources.
- No Pre-emption: Resources cannot be forcibly taken from a process; they must be released voluntarily.
- Circular Wait: A circular chain of processes exists, each waiting for a resource held by the next.

3.2 Deadlock Handling Strategies

Operating systems can handle deadlock through three broad strategies:

- Prevention: Structurally eliminate one of the four necessary conditions.
- Avoidance: Dynamically assess each resource request and only grant it if the system remains in a safe state. The Banker's Algorithm implements this strategy.
- Detection and Recovery: Allow deadlock to occur, detect it, and then recover by terminating or rolling back processes.

3.3 Safe State and Safe Sequence

A system is in a safe state if there exists at least one safe sequence — an ordering of all processes such that each process's remaining resource needs can be satisfied by the currently available resources plus the resources held by all previously completed processes in the sequence. If no such sequence exists, the system is in an unsafe state, which may lead to deadlock.

3.4 Resource Allocation Concepts

The algorithm operates on the following data structures:

- Available: A vector representing the number of available instances of each resource type.
- Maximum: A matrix where $Maximum[\ i\ ][\ j\ ]$ represents the maximum number of instances of resource j that process i may ever request.
- Allocation: A matrix where $Allocation[\ i\ ][\ j\ ]$ represents the number of instances of resource j currently allocated to process i.
- Need: A derived matrix where $Need[\ i\ ][\ j\ ] = Maximum[\ i\ ][\ j\ ] - Allocation[\ i\ ][\ j\ ]$, representing the remaining resources process i may still request.

4. Methodology and Implementation

4.1 Project Approach

The project was developed using a strict separation of concerns, all algorithm logic was implemented and tested independently of the user interface. Once verified correct, the Tkinter-based GUI was layered on top. This approach ensured that bugs could be isolated to either the logic or the UI layer, greatly simplifying debugging.

Development followed an incremental approach: the Process class was built first, followed by the Safety Algorithm, then the System class integrating both, and finally the Tkinter UI.

4.2 Tools and Technologies

Technology	Purpose
Python 3.14	Core implementation language
Tkinter	Built-in Python GUI library for the simulator interface
ttk (themed Tkinter)	Styled widgets — labels, buttons, treeviews, comboboxes
OOP (Classes)	Process and System modelled as objects with attributes and methods

4.3 Architecture Overview

The project consists of four Python modules:

- *Process.py* — Defines the process class representing an individual OS process.
- *Safety.py* — Implements the Safety Algorithm and the step-by-step walkthrough function.
- *System.py* — Implements the system class that manages the overall resource state and handles requests.
- *ui.py* — Implements the full Tkinter GUI, building on top of the System class.

5. Code Explanation

5.1 Process Class (Process.py)

The process class models a single OS process. Each process stores its PID, maximum resource demand (pmax), and current allocation. The need attribute is implemented as a Python @property meaning it is computed on demand as pmax minus allocation, rather than stored. This design ensures that need is always consistent with the current allocation without requiring manual updates.

```
class process:
    def __init__(self, pid, pmax, allocation):
        self.pid = pid
        self.pmax = pmax
        self.allocation = allocation
        self.is_finished = False

    @property
    def need(self):
        return [self.pmax[i] - self.allocation[i]
                for i in range(len(self.pmax))]
```

Key design decision: storing need as a computed property prevents synchronisation bugs. If allocation changes after a request is granted, need automatically reflects the new value without any additional code.

5.2 Safety Algorithm (Safety.py)

The safety function implements the classical Safety Algorithm. It operates on a copy of the process list and the available vector to avoid mutating the actual system state. The algorithm iterates in a while loop; in each pass, it scans all unfinished processes and checks whether their need can be satisfied by the current available resources (using element-wise comparison via Python's all() function). When a process is satisfied, its allocation is returned to available, it is marked finished, and the scan restarts. If a full pass completes with no progress, the algorithm terminates the system is in an unsafe state.

```
def safety(Processes, available):
    work = Processes.copy()
    sequence = []
    while True:
        progress = False
        for p in work:
            if all(p.need[j] <= available[j]
                   for j in range(len(available))):
                available = [available[j] + p.allocation[j]
                             for j in range(len(available))]
                p.is_finished = True
                sequence.append(p.pid)
                work.remove(p)
                progress = True
                break
        if not progress:
            break
    return is_work_safe(work), sequence
```

The function returns both a boolean (safe or not) and the safe sequence a list of PIDs in the order they can complete. The safe sequence is surfaced in the UI to aid understanding.

A second function, *safety_walkthrough*, is also implemented. It records each step of the algorithm which process was checked, whether it fit, the available vector before and after and returns these as a list of dictionaries. This powers the step-by-step walkthrough panel in the UI.

5.3 System Class (System.py)

The system class holds the system's available resources and list of process objects. Its primary method is request(), which implements the Resource-Request Algorithm:

1. Locate the requesting process by PID.
2. Check 1: Verify the request does not exceed the process's remaining need.
3. Check 2: Verify the request does not exceed the currently available resources.
4. Tentatively grant: subtract request from available, add to process allocation.
5. Run the Safety Algorithm on this hypothetical new state.
6. If safe: keep changes and return True. If unsafe: roll back and return False.

```
def request(self, pid, req_vector):
    process = next((p for p in self.processes
                    if p.pid == pid), None)
    if process is None: return False

    if not all(req_vector[i] <= self.available[i]
               for i in range(len(req_vector))): return False
    if not all(req_vector[i] <= process.need[i]
               for i in range(len(req_vector))): return False

    old_available = self.available.copy()
    old_allocation = process.allocation.copy()

    self.available = [self.available[i] - req_vector[i]
                      for i in range(len(req_vector))]
    process.allocation = [process.allocation[j] + req_vector[j]
                          for j in range(len(req_vector))]
```

```
    is_safe, _ = safety(self.processes, self.available[:])
    if is_safe: return True
    self.available = old_available
    process.allocation = old_allocation
    return False
```

5.4 User Interface (ui.py)

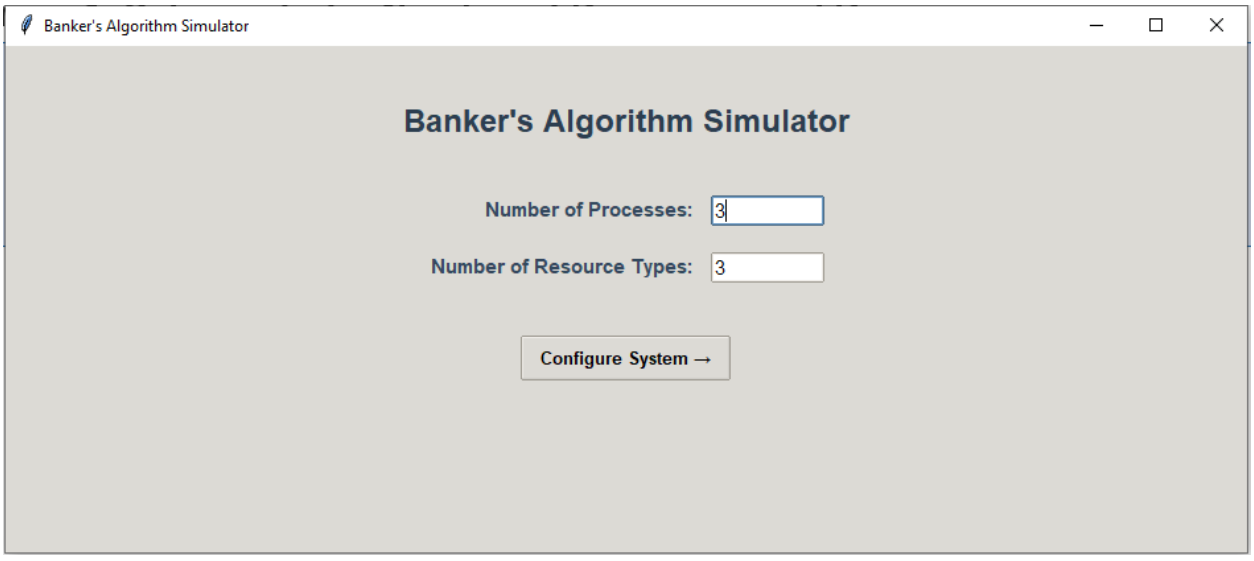
The GUI is implemented as a single BankersApp class using Tkinter and the ttk themed widget set. It is structured as three sequential screens:

- Screen 1: Setup: User inputs the number of processes and resource types.
- Screen 2: Data Entry: Dynamically generated grid of entry fields for Maximum and Allocation matrices, plus the Available vector. Input is validated before proceeding.
- Screen 3: Main Simulator: Displays the current system state in a Treeview table, shows safety status and safe sequence, provides a request submission panel with a request log, and renders the step-by-step Safety Algorithm walkthrough.

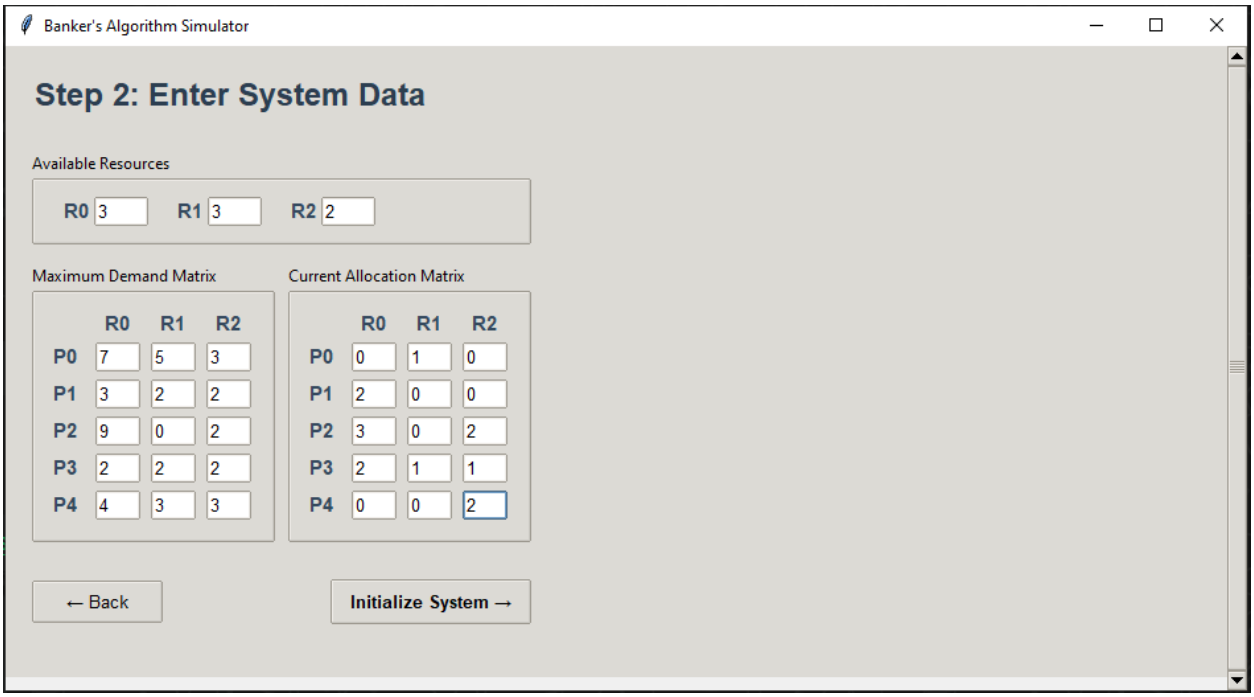
The walkthrough panel is a notable feature — it shows each pass of the Safety Algorithm row by row, colour-coded green for processes that fit and grey for those that do not, with a final conclusion row indicating SAFE or UNSAFE with the resulting sequence.

6. Screenshots of the Simulation

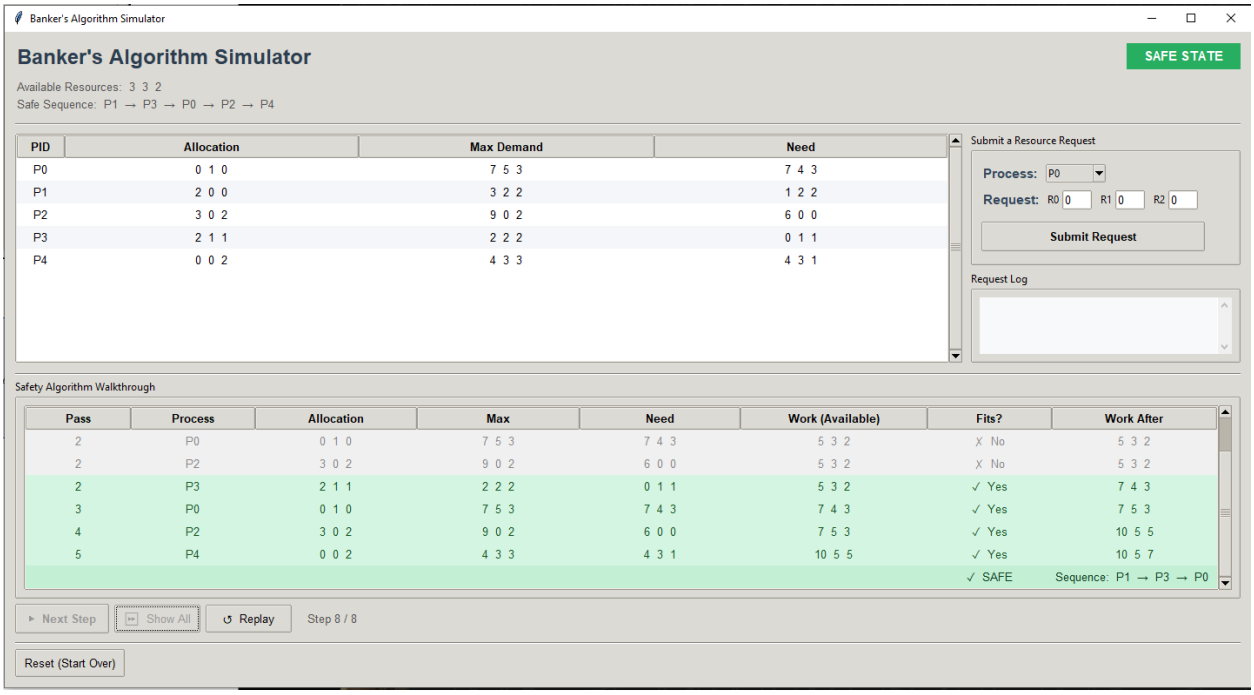
Screen 1: Setup — Enter Number of Processes and Resources



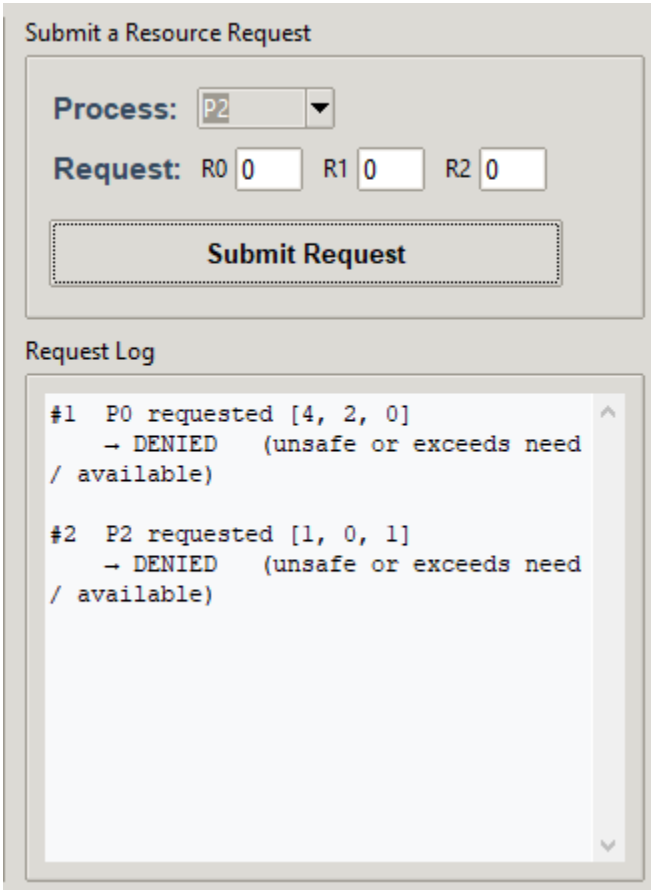
Screen 2: Data Entry — Maximum and Allocation Matrix Input



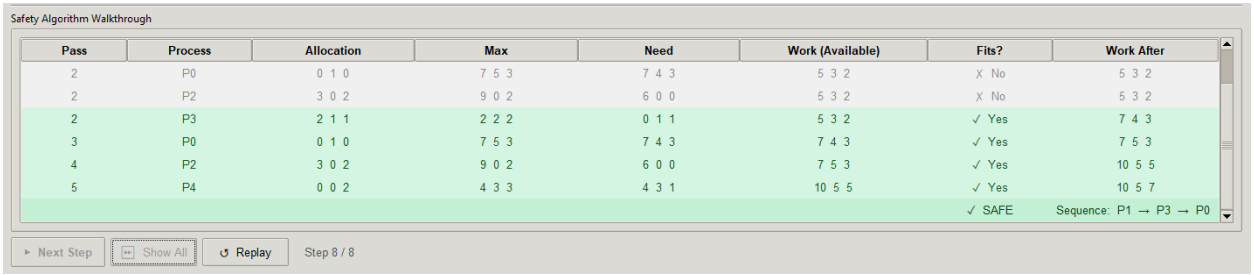
Screen 3: Main Simulator — System State Display

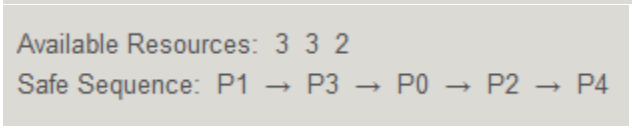


Screen 3: Request Submission and Log



Screen 3: Safety Algorithm Walkthrough





7. Challenges Faced and How They Were Resolved

Challenge	Resolution
List comparison with <= operator	Python's <= operator on lists performs lexicographic comparison, not element-wise. Using p.need <= available silently produced wrong results. Replaced with all(p.need[j] <= available[j] for j in range(len(available))) for correct element-wise checking.
@property defined inside __init__	The need property was initially indented inside the constructor, making it a local function that disappeared after initialisation. Moving it to class level resolved the issue.
Infinite loop in Safety Algorithm	Without a progress flag, the while loop had no exit condition for the unsafe case. Added a boolean progress variable reset before each pass; if a full pass completes with no process being satisfied, the loop breaks.
List mutation during iteration	Calling work.remove(p) inside a for loop over work caused processes to be skipped. Resolved by breaking immediately after removing a process and restarting the for loop from the beginning via the outer while loop.
Rollback of system state after unsafe request	When a tentatively granted request resulted in an unsafe state, the original available and allocation values needed to be restored. Solved by saving copies before the grant and reassigning on rollback.
Dynamic Tkinter grid generation	The number of input fields for the matrix depends on user-provided values, so widgets had to be created programmatically. Used nested loops to generate and store Entry widgets in 2D lists for later retrieval.

8. Conclusion and Observations

This project successfully implements and visualises the Banker's Algorithm in an interactive Python-based simulator. The simulator demonstrates how an operating system can dynamically manage resource allocation to prevent deadlock while allowing processes to acquire resources incrementally at runtime.

Key observations from the implementation:

- The separation of Need as a computed property from stored Allocation is a critical design choice, it eliminates an entire class of synchronisation bugs.
- The Safety Algorithm's behaviour is most clearly understood through the step-by-step walkthrough, which makes the concept of a safe sequence tangible and visible.
- The rollback mechanism in the Request Algorithm ensures that tentative grants never corrupt the system state, making the simulator robust even when unsafe requests are submitted.
- The Banker's Algorithm has practical limitations: it requires processes to declare their maximum needs upfront, assumes a fixed process count, and does not scale well to modern dynamic systems. However, as a teaching model for deadlock avoidance, it remains highly effective.

Building this project deepened our understanding of how theoretical OS concepts translate into concrete code, particularly how data structure choices (lists vs stored values), loop design (progress detection, early exit), and state management (copy vs reference) directly impact correctness.

References

- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley.
- Dijkstra, E. W. (1965). Co-operating Sequential Processes. Technical Report EWD-123, Eindhoven University of Technology.
- Python Software Foundation. (2024). Python 3 Documentation — tkinter: Python interface to Tcl/Tk. https://docs.python.org/3/library/tkinter.html
- Unwired Learning. (2024). Banker's Algorithm Guide. https://unwiredlearning.com/blog/bankers-algorithm-guide